

Homework 3: Visualizing Data Using Matplotlib

After going through the lecture notebooks, work on the homework below.

Question

Your task is to perform a **Pareto analysis** (decile analysis) on a dataset from MyAnimeList [1], a popular anime database containing information on over 17,000 anime titles.

Using the downloaded and preprocessed data (see Section 2), `anime_data`, write a function that performs the following steps:

1. Sort Anime by the Given Metric:

- Sort the anime in descending order based on the values in the column specified by `metric_column`.

2. Group Anime into n Equal Parts:

- Without changing the sorted order, divide the anime into n groups of equal size.
 - n is a natural number (positive integer).
 - Use `pd.qcut()` on the rank of the metric values (with `method='first'`) to handle grouping. This avoids issues with duplicate values.

3. Calculate Proportion for Each Group:

- For each group, calculate what percentage of the **overall total** (of `metric_column`) that group represents.
 - This is known as **decile analysis** when $n = 10$, but works for any valid n .

4. Sort Groups and Label:

- Sort the groups in descending order of their proportion.
- Label them as "Group 1", "Group 2", ..., "Group n ".
- Return the result as a `pandas.Series`.

Background:

The **Pareto Principle** states that roughly 80% of effects come from 20% of causes. In the anime context, a small number of highly popular anime may account for the vast majority of total viewership. This principle is widely used in business, marketing, and data analysis.

Inputs / Outputs:

```
homework(anime_data: pd.DataFrame, metric_column: str, n: int) -> pd.Series
```

Argument	Type	Description
<code>anime_data</code>	<code>pd.DataFrame</code>	Preprocessed anime dataset (see Section 2)
<code>metric_column</code>	<code>str</code>	Column name to analyze, e.g. 'Completed', 'Members'
<code>n</code>	<code>int</code>	Number of groups to divide into
Return	Type	Description
<code>result</code>	<code>pd.Series</code>	Proportions per group, sorted descending, index = "Group 1" ... "Group n ", values sum to 1.0

Example:

```
result = homework(anime_data, 'Completed', 5)
print(result)
```

```
Group 1    0.94XXXX
Group 2    0.04XXXX
Group 3    0.009XXX
Group 4    0.001XXX
Group 5    0.0004XX
dtype: float64
```

Note:

- Submit **only** the `homework()` function to Omnicampus.
- Erase `!!WRITE ME!!` before submitting.
- Do **NOT** include `import` statements in your submission.
- Do **NOT** modify the preprocessing cell (Section 2).
- Return type must be `pd.Series` with values summing to 1.0.

Deadline

Wed, May 13 (11:00 UTC)

1. Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import requests
import zipfile
import io
```

2. Downloading and Preprocessing the Dataset

We download the MyAnimeList database from GitHub and perform preprocessing.

Note: Do NOT modify the following two cells!

```
# Download MyAnimeList data from GitHub
url = 'https://github.com/Hernan4444/MyAnimeList-Database/archive/refs/heads/master.zip'
r = requests.get(url, stream=True)
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall()

# Load the anime data
anime_data_raw = pd.read_csv('MyAnimeList-Database-master/data/anime.csv')
print(f"Raw dataset shape: {anime_data_raw.shape}")
anime_data_raw.head()
```

Raw dataset shape: (17562, 35)

	MAL_ID	Name	Score	Genres	English name	Japanese name	Type	Episodes	Aired	Premiered	...	Score-10	Score-9	Score-8
0	1	Cowboy Bebop	8.78	Action, Adventure, Comedy, Drama, Sci-Fi, Space	Cowboy Bebop	カウボーイビバップ	TV	26	Apr 3, 1998 to Apr 24, 1999	Spring 1998	...	229170.0	182126.0	131625.0
1	5	Cowboy Bebop: Tengoku no Tobira	8.39	Action, Drama, Mystery, Sci-Fi, Space	Cowboy Bebop: The Movie	カウボーイビバップ 天国の扉	Movie	1	Sep 1, 2001	Unknown	...	30043.0	49201.0	49505.0
2	6	Trigun	8.24	Action, Sci-Fi, Adventure, Comedy, Drama, Shounen	Trigun	トライガン	TV	26	Apr 1, 1998 to Sep 30, 1998	Spring 1998	...	50229.0	75651.0	86142.0
				...		Witch			Jul 2,					

```
# Preprocessing - Do NOT modify this cell
columns_to_keep = ['MAL_ID', 'Name', 'Score', 'Type', 'Episodes',
                   'Members', 'Completed', 'Watching', 'Dropped', 'Popularity']
anime_data = anime_data_raw[columns_to_keep].copy()

# Remove rows where key metric columns have missing or zero values
anime_data = anime_data.dropna(subset=['Members', 'Completed', 'Watching'])
anime_data = anime_data[(anime_data['Members'] > 0) &
                        (anime_data['Completed'] > 0) &
```

```
(anime_data['Watching'] > 0)]
anime_data = anime_data.reset_index(drop=True)
print(f"Dataset shape after preprocessing: {anime_data.shape}")
anime_data.head()
```

Dataset shape after preprocessing: (16943, 10)

	MAL_ID	Name	Score	Type	Episodes	Members	Completed	Watching	Dropped	Popularity
0	1	Cowboy Bebop	8.78	TV	26	1251960	718161	105808	26678	39
1	5	Cowboy Bebop: Tengoku no Tobira	8.39	Movie	1	273145	208333	4143	770	518
2	6	Trigun	8.24	TV	26	558913	343492	29113	13925	201
3	7	Witch Hunter Robin	7.27	TV	26	94683	46165	4300	5378	1467
4	8	Bouken Ou Beet	6.98	TV	52	13224	7314	642	1108	4369

3. Solution

Write your solution below. Submit **only** the `homework()` function to Omnicampus.

```
# Submit only this function to Omnicampus.
# Don't forget to erase the "!!WRITE ME!!" before submitting!
def homework(anime_data, metric_column, n):
    my_result = 1+2
    return my_result
```

4. Test Your Solution

Run the test cell below to check that your `homework()` function does not contain import statements or data loading code before submitting.

Note: Do not modify the test code.

```
import ast
import inspect
import textwrap

def hw4_public_tests(homework_func):
    print("=== HW4 Public Tests ===")

    # Get the source code of the homework function
    source = textwrap.dedent(inspect.getsource(homework_func))
    tree = ast.parse(source)

    # ----- Test 1: No import statements -----
    print("Test 1: No import statements in function...", end=" ")
    for node in ast.walk(tree):
        if isinstance(node, (ast.Import, ast.ImportFrom)):
            print("NG")
        elif isinstance(node, ast.Import):
            names = ", ".join(alias.name for alias in node.names)
            print(f" Found import statement: '{names}'")
            print(" Remove all import statements before submitting.")
            return
    print("OK")

    # ----- Test 2: No data loading code -----
    print("Test 2: No data loading code in function...", end=" ")
    blocked_calls = {"read_csv", "read_excel", "read_json", "read_html",
                     "get", "post", "ZipFile", "extractall", "urlopen"}
    for node in ast.walk(tree):
        if isinstance(node, ast.Call):
            func_name = ""
            if isinstance(node.func, ast.Attribute):
                func_name = node.func.attr
            elif isinstance(node.func, ast.Name):
                func_name = node.func.id
            if func_name in blocked_calls:
                print("NG")
```

```

        print(f" Found data loading call: '{func_name}()'")
        print(" Do not include data downloading/loading code in your function.")
        return

    print("OK")

    # ----- Test 3: No hardcoded file paths or URLs -----
    print("Test 3: No hardcoded file paths or URLs...", end=" ")
    for node in ast.walk(tree):
        if isinstance(node, ast.Constant) and isinstance(node.value, str):
            val = node.value
            if val.startswith("http") or ".csv" in val or ".zip" in val:
                print("NG")
                print(f" Found hardcoded path/URL: '{val[:60]}...'")
                print(" Do not include file paths or URLs in your function.")
                return

    print("OK")

    print("=== All Public Tests Passed ===")

hw4_public_tests(homework)

```

5. Visualizing the Result

Let's create a **Pareto chart** to visualize the result of your analysis.

Notice how the top groups account for the majority of the total — this is the Pareto Principle in action!

```

N = 10
data = homework(anime_data, 'Completed', N)

fig, ax1 = plt.subplots(figsize=(8, 5))

data_num = len(data)
cum_per = np.cumsum(data)

# Bar chart – proportion per group
ax1.bar(range(data_num), data, color='steelblue', edgecolor='white')
ax1.set_xticks(range(data_num))
ax1.set_xticklabels(data.index, rotation=45, ha='right')

# Line chart – cumulative proportion
ax2 = ax1.twinx()
ax2.plot(range(data_num), cum_per, color='k', marker='o', linewidth=2)
ax2.set_ylim([0, 1.05])
ax2.axhline(y=0.8, color='r', linestyle='--', alpha=0.5, label='80% line')
ax2.grid(True, which='both', axis='y', alpha=0.3)

ax1.set_xlabel('Groups')
ax1.set_ylabel('Proportion', color='steelblue')
ax2.set_ylabel('Cumulative Proportion', color='k')
ax2.legend(loc='center right')
plt.title('Pareto Chart – Anime Completions by Group')
plt.tight_layout()
plt.show()

```

Once you've confirmed all public tests pass, copy and paste your `homework()` function to Omnicampus and submit. If you see `1.0`, your answer is correct.

References

[1] Hernan4444. *MyAnimeList Database*. GitHub. <https://github.com/Hernan4444/MyAnimeList-Database>

